Summarize

Chapter 13. Contracts

Friday, April 15, 12:01



Addison met with Sydney over lunch in the cafeteria to chat about coordination between the Ticket Orchestrator and the services it integrated with for the ticket management workflow.

"Why not just use gRPC for all the communication? I heard it's really fast," said Sydney.

"Well, that's an implementation, not an architecture," Addison said. "We need to decide what types of contracts we want before we choose how to implement them. First, we need to decide between tight or loose contracts. Once we decide on the type, I'll leave it to you to decide how to implement them, as long as they pass our fitness functions."

"What determines what kind of contract we need?" Sydney said.

In Chapter 2, we began discussing the intersection of three important forces—communication, consistency, and coordination—and how to develop trade-offs for them. We modeled the intersectional space of the three forces in a joined three-dimensional space, shown again in Figure 13-1. In Chapter 12, we revisited these three forces with a discussion of the various communication styles and their trade-offs.

However much an architecture can discern a relationship like this one, some forces cut across the conceptual space and affect all of the other dimensions equally. If pursuing the visual three-dimensional metaphor, these cross-cutting forces act as an additional dimension, much as *time* is orthogonal to the three physical dimensions.

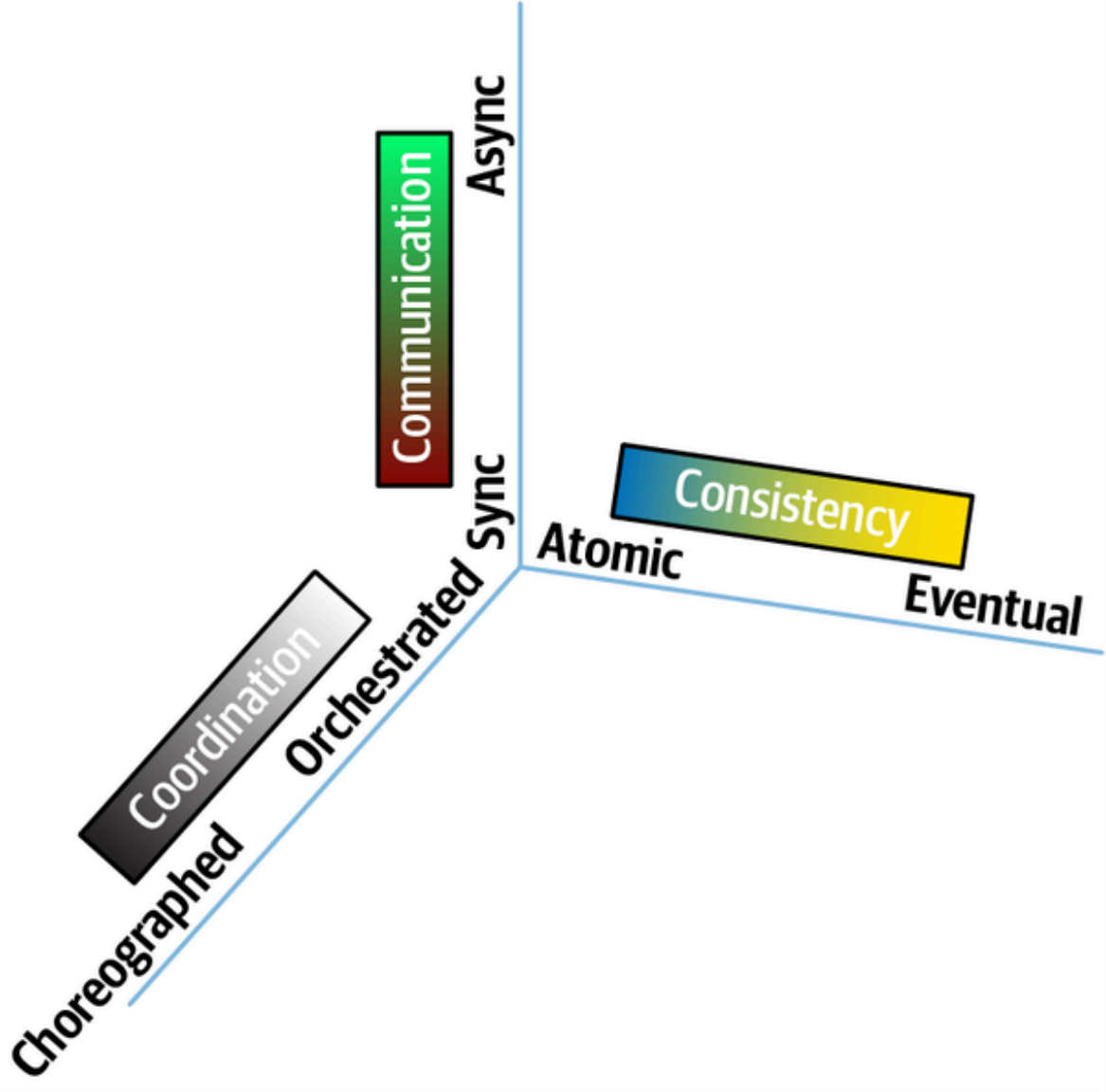


Figure 13-1. Three-dimensional intersecting space for messaging forces in distributed architectures

One constant factor in software architecture that cuts across and affects virtually every aspect of architect decision making is *contracts*, broadly defined as how disparate parts of an architecture connect with one another. The dictionary definition of a contract is as follows:

> *contract*
>
> A written or spoken agreement, especially one concerning employment, sales, or tenancy, that is intended to be enforceable by law.

In software, we use contracts broadly to describe things like integration points in architecture, and many contract formats are part of the design process of software development: SOAP, REST, gRPC, XMLRPC, and an alphabet soup of other acronyms. However, we broaden that definition and make it more consistent:

> *hard parts contract*
>
> The format used by parts of an architecture to convey information or dependencies.

This definition of *contract* encompasses all techniques used to "wire together" parts of a system, including transitive dependencies for frameworks and libraries, internal and external integration points, caches, and any other communication among parts.

This chapter illustrates the effects of contracts on many parts of architecture, including static and dynamic quantum coupling, as well as ways to improve (or harm) the effectiveness of workflows.

Strict Versus Loose Contracts

Like many things in software architecture, contracts don't exist within a binary but rather on a broad spectrum, from strict to loose. Figure 13-2 illustrates this spectrum, using example contract types.

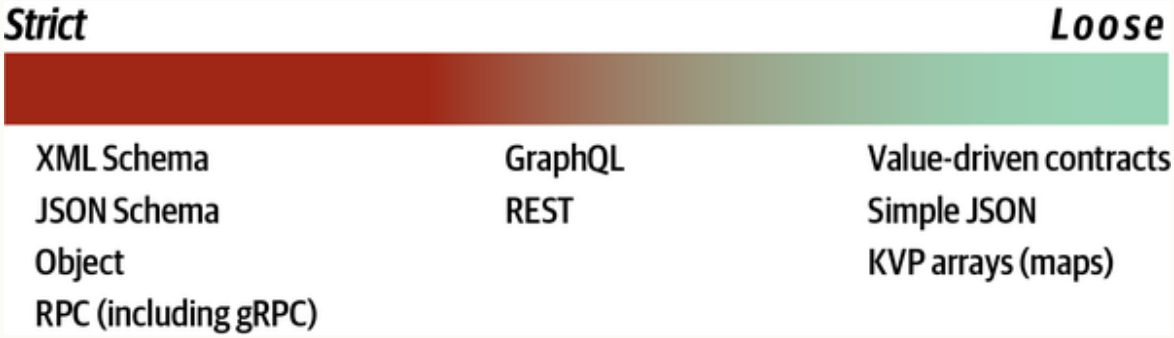


Figure 13-2. The spectrum of contract types, from strict to loose

A strict contract requires adherence to names, types, ordering, and all other details, leaving no ambiguity. An example of the strictest possible contract in software is a remote method call, using a platform mechanism such as RMI in Java. In that case, the remote call mimics an internal method call, matching name, parameters, types, and all other details.

Many strict contract formats mimic the semantics of method calls. For example, developers see a host of protocols that include some variation of the RPC, traditionally an acronym for *Remote Procedure Call*. gRPC is an example of a popular remote invocation framework that defaults to strict contracts.

Many architects like strict contracts because they model the identical semantic behavior of internal method calls. However, strict contracts create brittleness in integration architecture—something to avoid. As discussed in Chapter 8, something that is simultaneously changing frequently and used by several distinct architecture parts creates problems in architecture. Contracts fit that description because they form the glue within a distributed architecture: the more frequently they must change, the more rippling problems they cause for other services. However, architects aren't forced to use strict contracts and should do so only when advantageous.

Even an ostensibly loose format such as JSON offers ways to selectively add schema information to simple name-value pairs. Example 13-1 shows a strict JSON contract with schema information attached.

Example 13-1. Strict JSON contract

```
{
   "$schema": "http://json-schema.org/draft-04/schema#",
   "properties": {
    "acct": {"type": "number"},
    "cusip": {"type": "string"},
    "shares": {"type": "number", "minimum": 100}
  },
   "required": ["acct", "cusip", "shares"]
}
```

The first line references the schema definition we use and will validate against. We define three properties (`acct` , `cusip` , and `shares`), along with their types and, on the last line, which ones are required. This creates a strict contract, with required fields and types specified.

Examples of looser contracts include formats such as REST and GraphQL, very different formats but similar in demonstrating looser coupling than RPC-based formats. For REST, the architect models resources rather than method or procedure endpoints, making for less brittle contracts. For example, if an architect builds a RESTful resource that describes parts of an airplane to support queries about seats, that query won't break in the future if someone adds details about engines to the resource—adding more information doesn't break what's there.

Similarly, GraphQL is used by distributed architectures to provide read-only aggregated data rather than perform costly orchestration calls across a wide variety of services. Consider the two GraphQL representations in Examples 13-2 and 13-3, providing two different but capable views of the Profile contract.

Example 13-2. Customer Wishlist `Profile` representation

```
type Profile {
   name: String
}
```

Example 13-3. Customer `Profile` representation

```
type Profile {
   name: String
   addr1: String
   addr2: String
   country: String
   . . .
}
```

The concept of *profile* appears in both examples but with different values. In this scenario, the Customer Wishlist doesn't have internal access to the customer's name, only a unique identifier. Thus, it needs access to a Customer Profile that maps the identifier to the customer name. The Customer Profile, on the other hand, includes a large amount of information about the customer in addition to the name. As far as Wishlist is concerned, the only interesting thing in Profile is the name.

A common anti-pattern that some architects fall victim to is to assume that Wishlist might eventually need all the other parts, so the architects include them in the contract from the outset. This is an example of stamp coupling and an anti-pattern in most cases, because it introduces breaking changes where they aren't needed, making the architecture fragile yet providing little benefit. For example, if the Wishlist cares about only the customer name from Profile, but the contract specifies every field in Profile (just in case), then a change in Profile that Wishlist doesn't care about causes a contract breakage and coordination to fix. Keeping contracts at a "need to know" level strikes a balance between semantic coupling and necessary information without creating needless fragility in integration architecture.

At the far end of the spectrum of contract coupling lie extremely loose contracts, often expressed as name-value pairs in formats like YAML or JSON, as illustrated in Example 13-4.

Example 13-4. Name-value pairs in JSON

```
{
  "name": "Mark",
  "status": "active",
  "joined": "2003"
}
```

Nothing but the raw facts in this example! No additional metadata, type information, or anything else, just name-value pairs.

Using such loose contracts allows for extremely decoupled systems, often one of the goals in architectures, such as microservices. However, the looseness of the contract comes with trade-offs such as lack of contract certainty, verification, and increased application logic. We illustrate in "Contracts in Microservices" how architects resolve this problem by using contract fitness functions.

Trade-Offs Between Strict and Loose Contracts

When should an architect use strict contracts and when should they use looser ones? Like all the hard parts of architecture, no generic answer exists for this question, so it is important for architects to understand when each is most suitable.

Strict contracts

Stricter contracts have a number of advantages, including these:

Guaranteed contact fidelity

Building schema verification within contracts ensures exact adherence to the values, types, and other governed metadata. Some problem spaces benefit from tight coupling for contract changes.

Versioned

Strict contracts generally require a versioning strategy to support two endpoints that accept different values or to manage domain evolution over time. This allows gradual changes to integration points while supporting a selective number of past versions to make integration collaboration easier.

Easier to verify at build time

Many schema tools provide mechanisms to verify contracts at build time, adding a level of type checking for integration points.

Better documentation

Distinct parameters and types provide excellent documentation with no ambiguity.

Strict contracts also have a few disadvantages:

Tight coupling

By our general definition of coupling, strict contracts create tight coupling points. If two services share a strict contract and the contract changes, both services must change.

Versioned

This appears in both advantages and disadvantages. While keeping distinct versions allows for precision, it can become an integration nightmare if the team doesn't have a clear deprecation strategy or tries to support too many versions.

The trade-offs for strict contracts are summarized in Table 13-1.

TRADE-OFFS

Table 13-1. Trade-offs for strict contracts

Advantage	Disadvantage
Guaranteed contract fidelity	Tight coupling
Versioned	Versioned
Easier to verify at build time	
Better documentation	

Loose contracts

Loose contracts, such as name-value pairs, offer the least coupled integration points, but they too have trade-offs, as summarized in Table 13-2.

These are some advantages of loose contracts:

Highly decoupled

Many architects have a stated goal for microservices architectures that includes high levels of decoupling, and loose contracts provide the most flexibility.

Easier to evolve

Because little or no schema information exists, these contracts can evolve more freely. Of course, semantic coupling changes still require coordination across all interested parties—implementation cannot reduce semantic coupling—but loose contracts allow easier implementation evolution.

Loose contracts also have a few disadvantages:

Contract management

Loose contracts by definition don't have strict contract features, which may cause problems such as misspelled names, missing name-value pairs, and other deficiencies that schemas would fix.

Requires fitness functions

To solve the contract issues just described, many teams use consumer-driven contracts as an architecture fitness function to make sure that loose contracts still contain sufficient information for the contract to function.

TRADE-OFFS

Table 13-2. Trade-offs for loose contracts

Advantage	Disadvantage
Highly decoupled	Contract management
Easier to evolve	Requires fitness functions

For an example of the common trade-offs encountered by architects, consider the example of contracts in microservice architectures.

Contracts in Microservices

Architects must constantly make decisions about how services interact with one another, what information to pass (the semantics), how to pass it (the implementation), and how tightly to couple the services.

Coupling levels

Consider two microservices with independent transactionality that must share domain information such as *Customer Address*, shown in Figure 13-3.

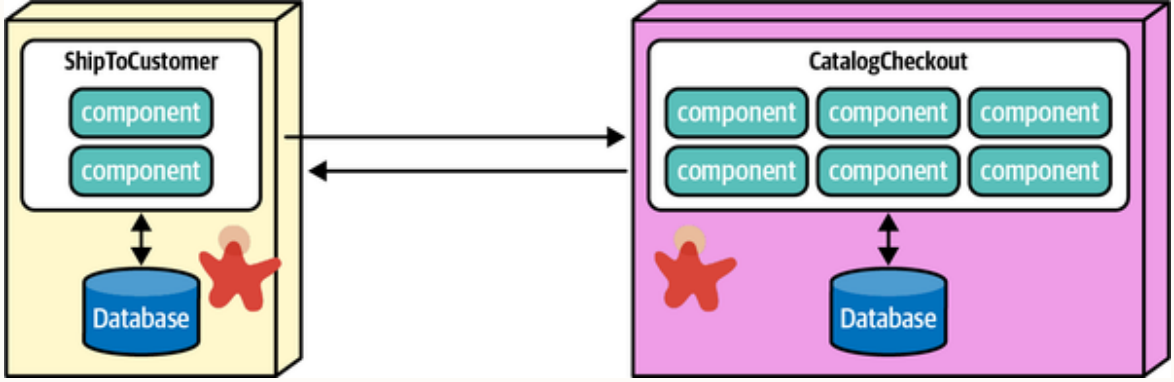


Figure 13-3. Two services that must share domain information about the customer

The architect could implement both services in the same technology stack and use a strictly typed contract, either a platform-specific remote procedure protocol (such as RMI) or an implementation-independent one like gRPC, and pass the customer information from one to another with high confidence of contract fidelity. However, this tight coupling violates one of the aspirational goals of microservices architectures, where architects try to create decoupled services.

Consider the alternative approach, where each service has its own internal representation of Customer, and the integration uses name-value pairs to pass information from one service to another, as illustrated in Figure 13-4.

Here, each service has its own bounded-context definition of Customer. When passing information, the architect utilizes name-value pairs in JSON to pass the relevant information in a loose contract.

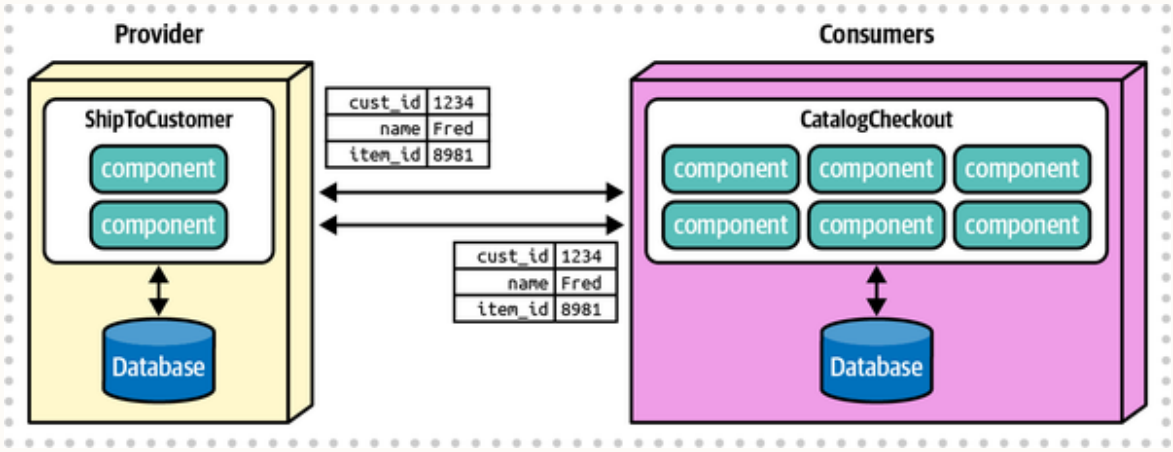


Figure 13-4. Microservices with their own internal semantic representation can pass values in simple messages

This loose coupling satisfies many of the overarching goals of microservices. First, it creates highly decoupled services modeled after bounded contexts, allowing each team to evolve internal representations as aggressively as needed. Second, it creates implementation decoupling. If both services start in the same technology stack, but the team in the second decides to move to another platform, it likely won't affect the first service at all. All platforms in common use can produce and consume name-value pairs, making them the lingua franca of integration architecture.

The biggest downside of loose contracts is contract fidelity—as an architect, how can I know that developers pass the correct number and type of parameters for integration calls? Some protocols, such as JSON, include schema tools to allow architects to overlay loose contracts with more metadata. Architects can also use a style of architect fitness function called a *consumer-driven contract*.

Consumer-driven contracts

A common problem in microservices architectures is the seemingly contradictory goals of loose coupling yet contract fidelity. One innovative approach that utilizes advances in software development is a *consumer-driven contract*, common in microservices architectures.

In many architecture integration scenarios, a service decides what information to emit to other integration partners (a *push* model—the service provider pushes a contract to consumers). The concept of a consumer-driven contract inverses that relationship into a *pull* model; here, the consumer puts together a contract for the items they need from the provider, and passes the contract to the provider, who includes it in their build and keeps the contract test green at all times. The contract encapsulates the information the consumer needs from the provider. This may work for a network of interlocking requests that the Provider must honor, as illustrated in Figure 13-5.

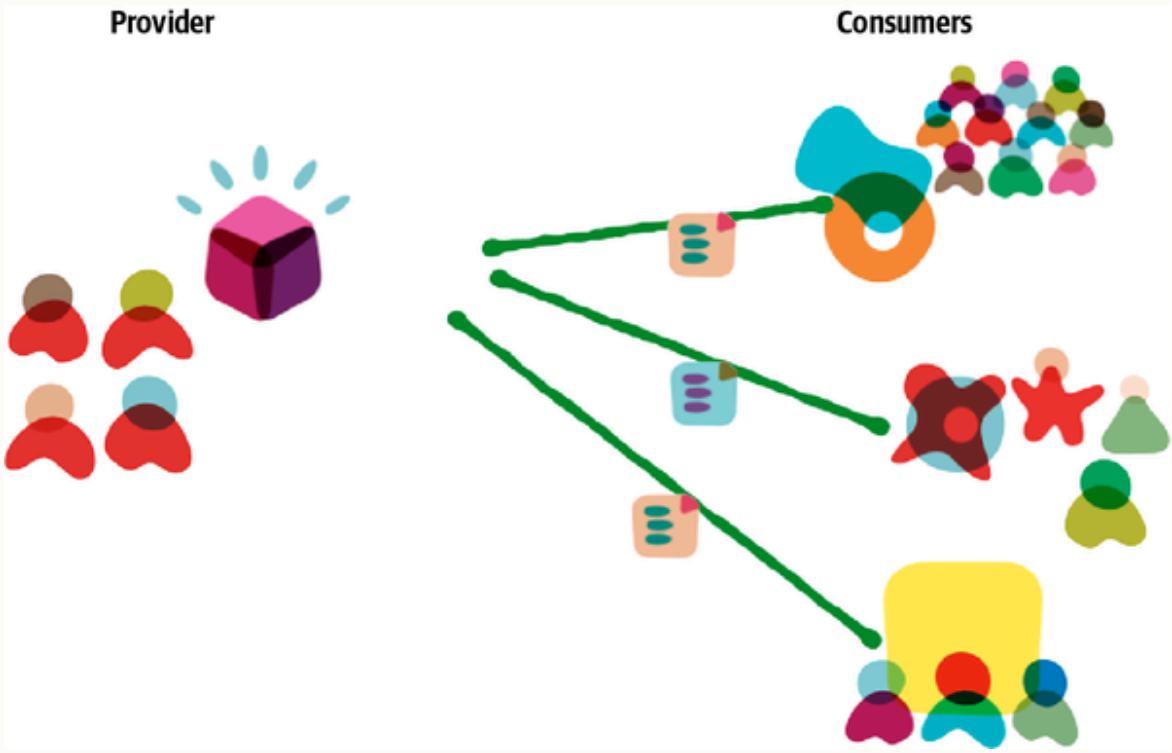


Figure 13-5. Consumer-driven contracts allow the provider and consumers to stay in sync via automated architectural governance

In this example, the team on the left provides bits of (likely) overlapping information to each of the consumer teams on the right. Each consumer creates a contract specifying required information and passes it to the provider, who includes their tests as part of a con-

tinuous integration or deployment pipeline. This allows each team to specify the contract as strictly or loosely as needed while guaranteeing contract fidelity as part of the build process. Many consumer-driven contract testing tools provide facilities to automate build-time checks of contracts, providing another layer of benefit similar to stricter contracts.

Consumer-driven contracts are quite common in microservices architecture because they allow architects to solve the dual problems of loose coupling and governed integration. Trade-offs of consumer-driven contracts are shown in Table 13-3.

Advantages of consumer-driven contracts are as follows:

Allow loose contract coupling between services

Using name-value pairs is the loosest possible coupling between two services, allowing implementation changes with the least chance of breakage.

Allow variability in strictness

If teams use architecture fitness functions, architects can build stricter verifications than typically offered by schemas or other type-additive tools. For example, most schemas allow architects to specify things like numeric type but not acceptable ranges of values. Building fitness functions allows architects to build as much specificity as they like.

Evolvable

Loose coupling implies evolvability. Using simple name-value pairs allows integration points to change implementation details without breaking the semantics of the information passed between services.

These are disadvantages of consumer-driven contracts:

Require engineering maturity

Architecture fitness functions are a great example of a capability that really works well only when well-disciplined teams have good practices and don't skip steps. For example, if all teams run continuous integration that includes contract tests, then fitness functions provide a good verification mechanism. On the other hand, if many teams ignore failed tests or are not timely in running contract tests, integration points may be broken in architecture longer than desired.

Two interlocking mechanisms rather than one

Architects often look for a single mechanism to solve problems, and many of the schema tools have elaborate capabilities to create end-to-end connectivity.

However, sometimes two simple interlocking mechanisms can solve the problem more simply. Thus, many architects use the combination of name-value pairs and consumer-driven contracts to validate contracts. However, this means that teams require two mechanisms rather than one.

The architect's best solution for this trade-off comes down to team maturity and decoupling with loose contracts versus complexity plus certainty with stricter contracts.

TRADE-OFFS

Table 13-3. Trade-offs for consumer-driven contracts

Advantage	Disadvantage
Allows loose contract coupling between services	Requires engineering maturity
Allows variability in strictness	Two interlocking mechanisms rather than one
Evolvable	

Stamp Coupling

A common pattern and sometimes anti-pattern in distributed architectures is stamp coupling, which describes passing a large data structure between services, but each service interacts with only a small part of the data structure. Consider the example of four services shown in Figure 13-6.

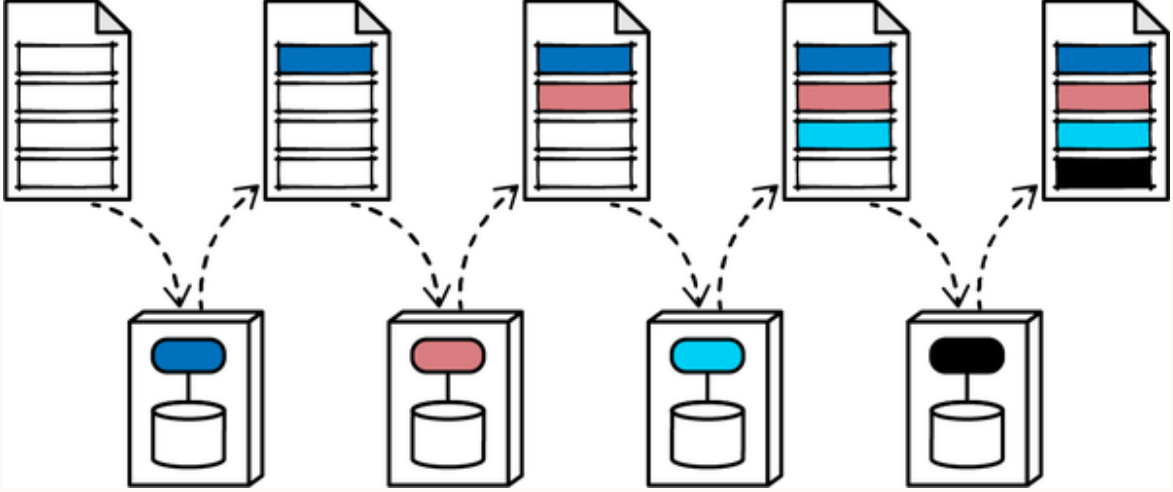


Figure 13-6. Stamp coupling between four services

Each service accesses (either reads, writes, or both) only a small portion of the data structure passed between each service. This pattern is common when an industry-standard document format exists, typically in XML. For example, the travel industry has a global standard XML document format that specifies details about travel itineraries. Several systems that work with travel-related services pass the entire document around, updating only their relevant sections.

Stamp coupling, however, is often an accidental anti-pattern, where an architect has over-specified the details in a contract that aren't needed or accidentally consumes far too much bandwidth for mundane calls.

Over-Coupling via Stamp Coupling

Going back to our Wishlist and Profile Services, consider tying the two together with a strict contract combined with stamp coupling, as illustrated in Figure 13-7.

In this example, even though the Wishlist Service needs only the name (accessed via a unique ID), the architect has coupled Profile's entire data structure as the contract, perhaps in a misguided effort for future proofing. However, the negative side effect of too much coupling in contracts is brittleness. If Profile changes a field that Wishlist doesn't care about, such as `state`, it still breaks the contract.

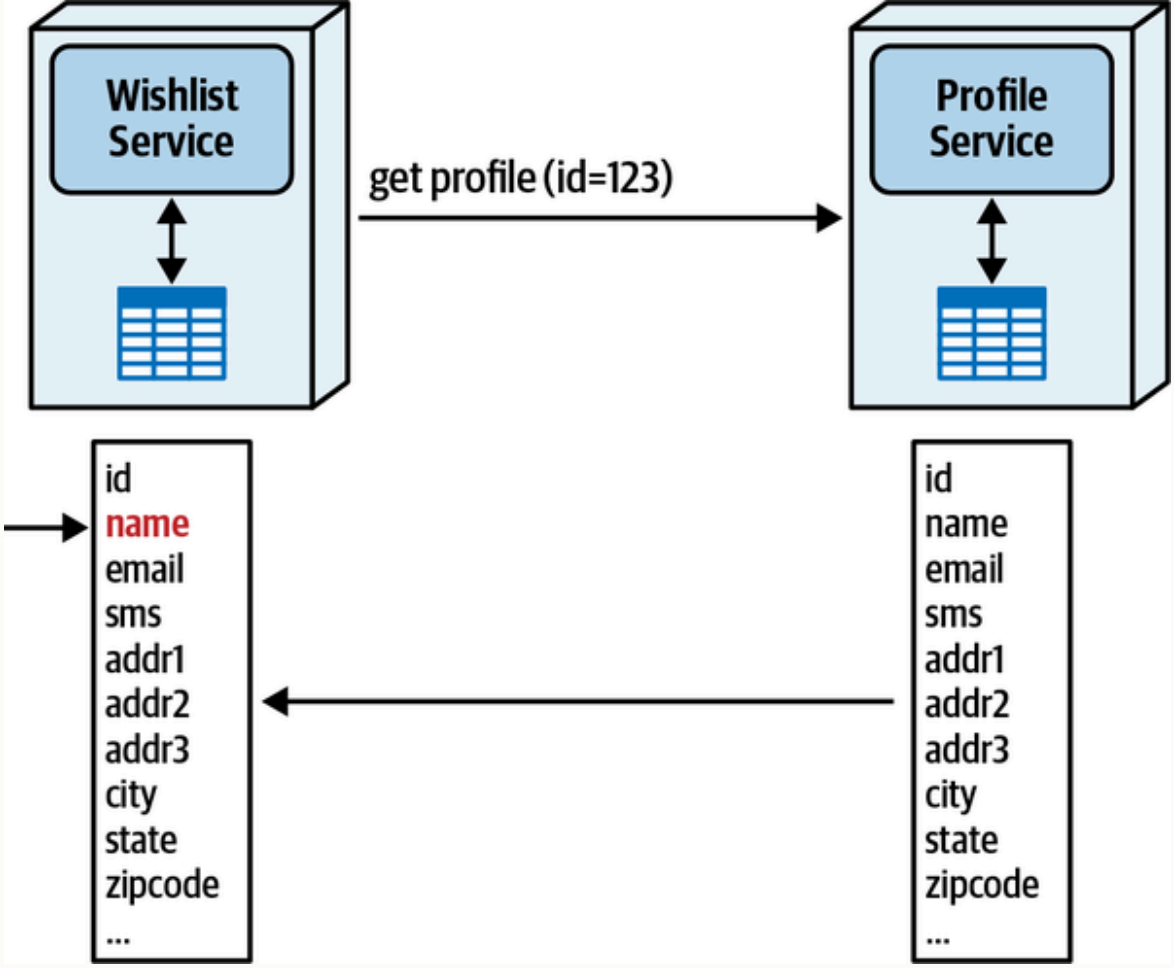


Figure 13-7. The Wishlist Service is stamp coupled to the Profile Service

Over-specifying details in contracts is generally an anti-pattern but easy to fall into when also using stamp coupling for legitimate concerns, including uses such as workflow management (see "Stamp Coupling for Workflow Management").

Bandwidth

The other inadvertent anti-pattern that some architects fall into is one of the famous fallacies of distributed computing: *bandwidth is infinite*. Architects and developers rarely have to consider the cumulative size of the number of method calls they make within a monolith because natural barriers exist. However, many of those barriers disappear in distributed architectures, inadvertently creating problems.

Consider the previous example for 2,000 requests per second. If each payload is 500 KB, then the bandwidth required for this single request equals 1,000,000 KB per second! This is obviously an egregious use of bandwidth for no good reason. Alternatively, if the coupling between Wishlist and Profile contained only the necessary information, `name`, the overhead changes to 200 *bytes* per second, for a perfectly reasonable 400 KB.

Stamp coupling can create problems when overused, including issues caused by coupling too tightly to bandwidth. However, like all things in architecture, it has beneficial uses as well.

Stamp Coupling for Workflow Management

In Chapter 12, we covered a number of dynamic quantum communication patterns, including several that featured the *coordination* style of *choreography*. Architects tend toward mediation for complex workflows for the many reasons we've delineated. However, what if other factors, such as scalability, drive an architect toward a solution that is both choreographed and complex?

Architects can use stamp coupling to manage the workflow state between services, passing both domain knowledge and workflow state as part of the contract, as illustrated in Figure 13-8.

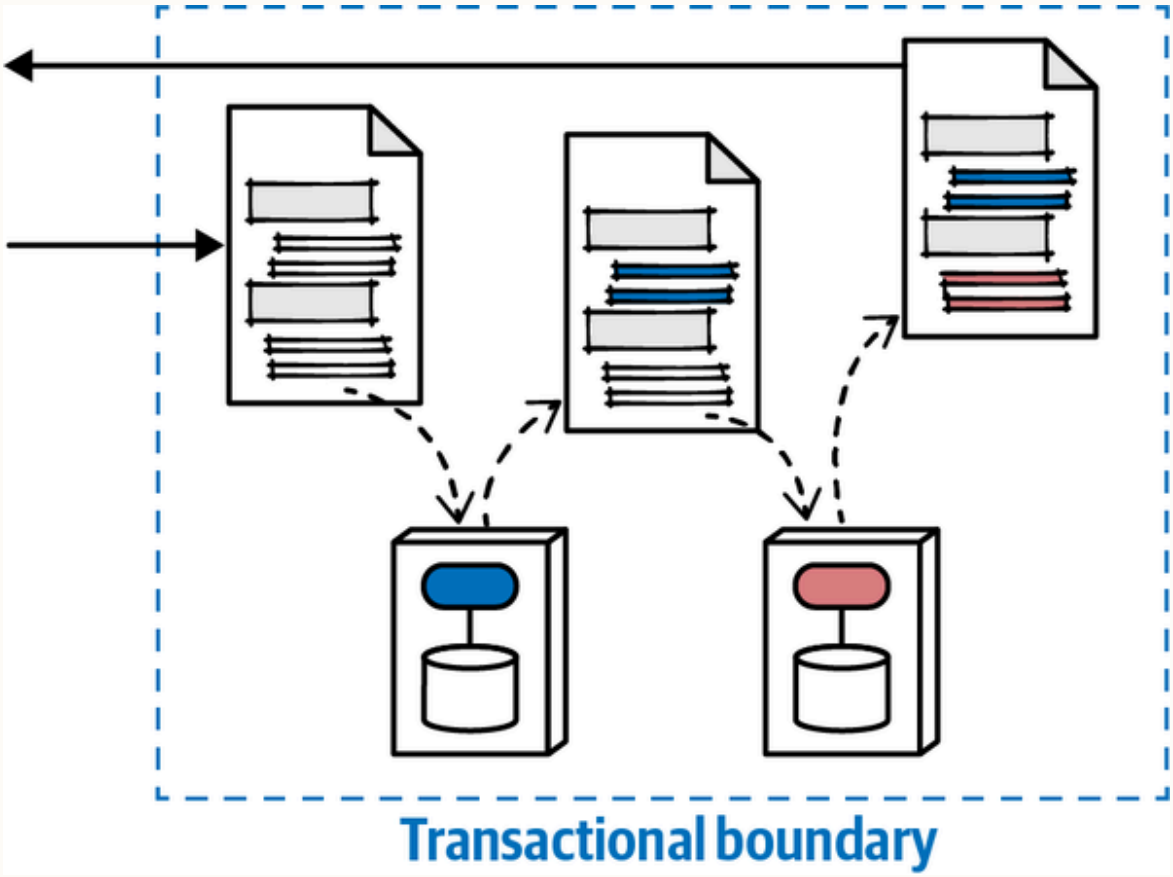


Figure 13-8. Using stamp coupling for workflow management

In this example, an architect designs the contract to include workflow information: status of the workflow, transactional state, and so on. As each domain service accepts the contract, it updates its portion of the contract and state for the workflow, then passes it along. At the end of the workflow, the receiver can query the contract to determine success or failure, along with status and information such as error messages. If the system needs to implement transactional consistency throughout, then domain services should rebroadcast the contract to previously visited services to restore atomic consistency.

Using stamp coupling to manage workflow does create higher coupling between services than nominal, but the semantic coupling must go somewhere—remember, an architect cannot reduce semantic coupling via implementation. However, in many cases, switching to

choreography can improve throughput and scalability, making the choice of stamp coupling over mediation an attractive one. Table 13-4 shows the trade-offs for stamp coupling.

TRADE-OFFS

Table 13-4. Trade-offs for stamp coupling

Advantage	Disadvantage
Allows complex workflows within choreographed solutions	Creates (sometimes artificially) high coupling between collaborators
	Can create bandwidth issues at high scale

Sysops Squad Saga: Managing Ticketing Contracts

Tuesday, May 10, 10:10



Sydney and Addison met again in the cafeteria over coffee to discuss the contracts in the ticket management workflow.

Addison said, "Let's look at the workflow under discussion, the ticket management workflow. I've sketched out the types of contracts we should use, and wanted to run it by you to make sure I wasn't missing anything. It's illustrated in Figure 13-9."

"The contracts between the orchestrator and the two ticket services, Ticket Management and Ticket Assignment, are tight; that information is highly semantically coupled and likely to change together," Addison said. "For example, if we add new types of things to manage, the assignment must sync up. The Notification and Survey Service can be much looser—the information changes more slowly, and doesn't benefit from brittle coupling."

Sydney said, "All those decisions make sense—but what about the contract between the orchestrator and the Sysops Squad expert application? It seems that would need as tight a contract as assignment."

"Good catch—nominally, we would like the contract with the mobile application to match ticket assignment. However, we deploy the mobile application through a public app store,

and their approval process sometimes takes a long time. If we keep the contracts looser, we gain flexibility and slower rate of change."

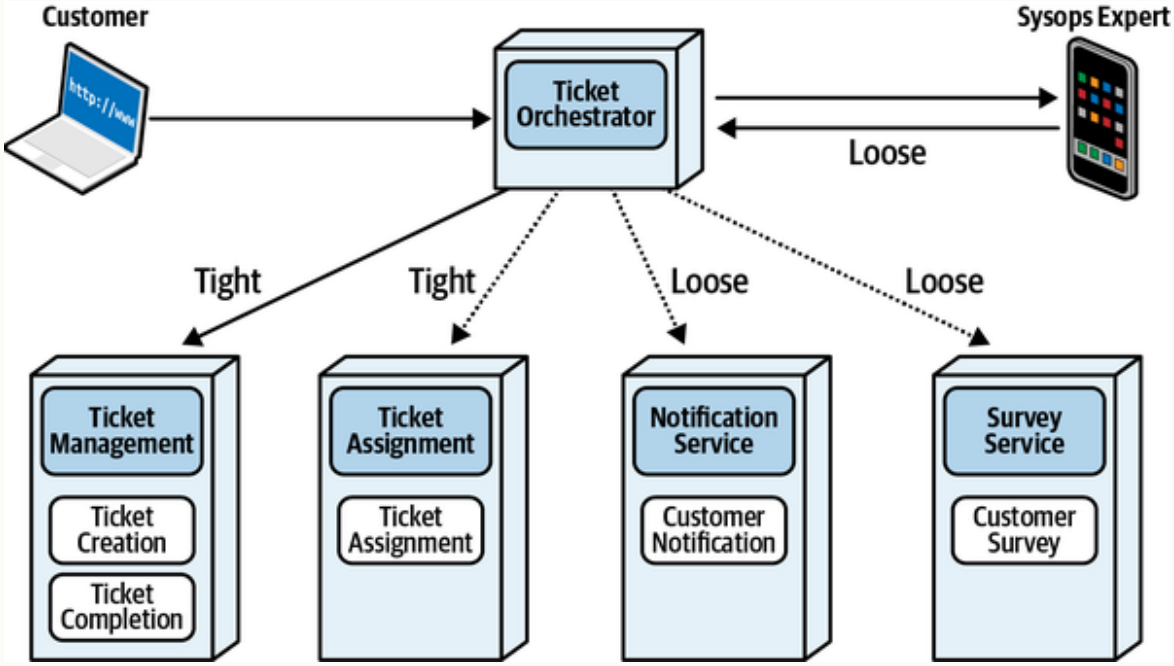


Figure 13-9. Types of contracts between collaborators in the ticket management workflow

They both wrote an ADR for this:

> *ADR: Loose Contract for Sysops Squad Expert Mobile Application*
>
> *Context*
> *The mobile application used by Sysops Squad experts must be deployed through the public app store, imposing delays on the ability to update contracts.*
>
> *Decision*
> *We will use a loose, name-value pair contract to pass information to and from the orchestrator and the mobile application.*
>
> *We will build an extension mechanism to allow temporary extensions for short-term flexibility.*
>
> *Consequences*
> *The decision should be revisited if the app store policy allows for faster (or continuous) deployment.*
>
> *More logic to validate contracts must reside in the orchestrator and mobile application.*

Previous chapter

< 12. Transactional Sagas

Next chapter

14. Managing Analytical Data >